Unit IV

String Handling and Java 8 Features

String Class

In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string. For example:

```
char[] ch={'C','O','C','S','I','T'};
String s=new String(ch);
```

is same as:

```
String s="COCSIT";
```

Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create a string object. String class is immutable class. It means we can't modify once string is created. And if we try to modify it will create new string object.
There are two ways to create String object:

- By string literal
- By new keyword

1. String Literal
Java String literal is created by using double quotes.

For Example:

```
String s="welcome";
```

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool.

For example:

String s1="Welcome";
String s2="Welcome"; //It doesn't create a new instance

In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Welcome" in string constant pool that is why it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance. This makes Java more memory efficient.

2. By new keyword

String s=new String("Welcome"); //creates two objects and one reference variable

In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

String Methods in Java

1. int length()
   This method returns the length of this string.
2. char charAt(int index)
   This method returns the char value at the specified index.
3. String concat(String str)
   This method concatenates the specified string to the end of this string.
4. boolean equals(Object anObject)
   This method compares this string to the specified object.
5. int indexOf(int ch)
   This method returns the index within this string of the first occurrence of the specified character.
6. String replace(char oldChar, char newChar)
   This method returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.
7. String substring(int beginIndex)
   This method returns a new string that is a substring of this string.

8. String substring(int beginIndex, int endIndex)

This method returns a new string that is a substring of this string.

9. String toLowerCase()

This method converts all of the characters in this String to lower case using the rules of the default locale.

10. String toUpperCase()

This method converts all of the characters in this String to upper case using the rules of the default locale.

11. String trim()

This method returns a copy of the string, with leading and trailing whitespace omitted.

Example for using String methods

```
public class DemoString
{
        public static void main(String[] args)
        {
                String s1="COCSIT";
                String s2="Latur";
                String s3="cocsit";

                System.out.println("Length of s1 "+s1.length());
                System.out.println("Accessing single char "+s1.charAt(3));
                System.out.println("Concating two strings "+s1.concat(s2));
                System.out.println("Accessing multiple chars "+s1.substring(3));
                System.out.println("s1 equals s3 "+s1.equals(s3));
                System.out.println("To Lower Case "+s1.toLowerCase());
                System.out.println("To Upper Case "+s2.toUpperCase());
        }
}
```

Output

```
Length of s1 6
Accessing single char S
Concating two strings COCSITLatur
Accessing multiple chars SIT
s1 equals s3 false
```

To Lower Case cocsit

To Upper Case LATUR

StringBuffer Class

StringBuffer is a class in Java that represents a mutable sequence of characters. It provides an alternative to the immutable String class, allowing you to modify the contents of a string without creating a new object every time.

Constructors of StringBuffer class

1. StringBuffer()

   It creates an empty String buffer with the initial capacity of 16. Example

   StringBuffer sb1=new StringBuffer();

2. StringBuffer(String str)

   It creates a String buffer with the specified string. Example

   StringBuffer sb1=new StringBuffer("COCSIT");

3. StringBuffer(int capacity)

   It creates an empty String buffer with the specified capacity as length. Example

   StringBuffer sb1=new StringBuffer(10);

Methods of StringBuffer class

We can use four common methods from String class. These common methods are length(), charAt(), substring() and indexOf(). Besides these common methods following are the methods of StringBuffer class.

1. append(String str)

   The append() method concatenates the given argument with this string.

2. insert(int index, String str)

   The insert() method inserts the given string with this string at the given position.

3. delete(int index, int index)

   The delete() method of the StringBuffer class deletes the string from the specified beginIndex to endIndex-1.

4. reverse()

   The reverse() method of the StringBuilder class reverses the current string.

5. capacity()

The capacity() method of the StringBuffer class returns the current capacity of the buffer. The default capacity of the buffer is 16. If the number of characters increases from its current capacity, it increases the capacity by (oldcapacity*2)+2.
For instance, if your current capacity is 16, it will be (16*2)+2=34.

Example for using StringBuffer methods

```
public class DemoStringBuffer
{
    public static void main(String[] args)
    {
        StringBuffer s = new StringBuffer("COCSIT");
        s.append(" Latur");
        System.out.println("String after append= "+s);
        int p = s.length();
        int q = s.capacity();
        System.out.println("Length of string GeeksforGeeks= "+ p);
        System.out.println("Capacity of string GeeksforGeeks= " + q);
        s.insert(6, " College");
        System.out.println("String after insert= "+s);
        s.reverse();
        System.out.println("String after reverse= "+s);
    }
}
```

Output
String after append= COCSIT Latur
Length of string GeeksforGeeks= 12
Capacity of string GeeksforGeeks= 22
String after insert= COCSIT College Latur
String after reverse= rutaL egelloC TISCOC

Introduction to IO streams

Java I/O (Input and Output) is used to process the input and produce the output. Java uses the concept of stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations. We can perform file handling in java by Java I/O API.

Stream

A stream is a sequence of data. In Java a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In java, 3 streams are created for us automatically. All these streams are attached with console.

1) System.out: standard output stream

2) System.in: standard input stream

3) System.err: standard error stream

Let's see the code to print output and error message to the console.

```
1. System.out.println("simple message");

2. System.err.println("error message");
```

Let's see the code to get input from console.

```
1. int i=System.in.read();          //returns ASCII code of 1st character

2. System.out.println((char)i);          //will print the character
```

Java brings various Streams with its I/O package that helps the user to perform all the input-output operations. These streams support all the types of objects, data-types, characters, files etc. to fully execute the I/O operations.

Types of Streams

1) Byte Streams (Binary Streams)

a) InputStream

This is an abstract class that describes stream input.

i)FileInputStream

This is used to read data from a file.

ii)BufferedInputStream

It is used for Buffered Input Stream.

iii)ObjectInputStream

It is used to read object from secondary device.

b) OutputStream

This is an abstract class that describe stream output.

i)FileOutputStream

This is used to write data into a file.

ii)BufferedOutputStream

This is used for Buffered Output Stream.

iii)ObjectOutputStream

It is used to write object into secondary device.

2) Character Streams (Unicode Char Streams)

a) Reader

This is an abstract class that define character stream input.

i)FileReader

This is an input stream that reads from file.

ii)BufferedReader

It is used to handle buffered input stream.

iii)InputStreamReader

This input stream is used to translate byte to character.

b) Writer

This is an abstract class that define character stream output.

i)FileWriter

This is used to output stream that writes to file.

ii)BufferedWriter

This is used to handle buffered output stream.

iii)OutputStreamWriter

This output stream is used to translate character to byte.

IO operations

Java I/O (Input/Output) operations are a crucial part of Java programming. They allow you to read and write data to and from various sources, such as files, the console, or network connections.

We will see example for reading data from one text file and writing data into another text file.

```
import java.io.File;

import java.io.FileReader;

import java.io.FileWriter;

import java.io.IOException;

public class DemoReadWrite

{

    public static void main(String[] args) throws IOException

    {
```

```
        File in=new File("src/abc.txt");
        File out=new File("src/xyz.txt");
        FileReader fin=new
        FileReader(in);
        FileWriter fout=new FileWriter(out);
        int c;
        while((c=fin.read())!=-1)
            fout.write(c);
        System.out.println("Data copied from abc.txt file to xyz.txt file");
    }
}
```

Output

Data copied from abc.txt file to xyz.txt file.

Object Serialization

Serialization in Java is a mechanism of writing the state of an object into a byte-stream. It is mainly used in Hibernate, RMI, JPA, EJB and JMS technologies.

The reverse operation of serialization is called deserialization where byte-stream is converted into an object. The serialization and deserialization process is platform-independent, it means you can serialize an object on one platform and deserialize it on a different platform.

For serializing the object, we call the writeObject() method of ObjectOutputStream class, and for deserialization we call the readObject() method of ObjectInputStream class.

We must have to implement the Serializable interface for serializing the object with java.io.Serializable interface. Serializable is a marker interface (has no data member and method). It is used to "mark" Java classes so that the objects of these classes may get a certain capability. The Cloneable and Remote are also marker interfaces.

Default Methods in Interface

Before Java 8, we could only declare abstract methods in an interface. However, Java 8 introduced the concept of default methods. Default methods are methods that can have a body. The most important use of default methods in interfaces is to provide additional functionality to a given type without breaking down the implementing classes.

Before Java 8, if a new method was introduced in an interface then all the implementing classes used to break. We would need to provide the implementation of that method in all the implementing classes.

However, sometimes methods have only single implementation and there is no need to provide their implementation in each class. In that case, we can declare that method as a default in the interface and provide its implementation in the interface itself.

Example of default methods

Let's understand the syntax of default methods through an example. Here, we have an interface with one abstract and one default method:

```
interface Arithmetic
{
        void add(int x,int y);
        default void sub(int x,int y)
        {
                System.out.println("Substraction = "+(x-y));
        }
}
class Operations implements Arithmetic
{
        public void add(int x, int y)
        {
                System.out.println("Addition = "+(x+y));
        }
}
public class DemoDefaultMethod
{
        public static void main(String[] args)
        {
```

```
            Arithmetic obj=new Operations();
            obj.add(45, 30);
            obj.sub(45, 30);
        }
}
```

Output
Addition = 75
Substraction = 15

Static Methods in Interface

Static methods were introduced in Java 8 to allow interfaces to define methods that belong to the interface itself, not to any instance of the interface. Java interface static method is similar to default method except that we can't override them in the implementation classes. This feature helps us in avoiding undesired results in case of poor implementation in implementation classes.

Important points about java interface static method:

- Java interface static method is part of interface, we can't use it for implementation class objects.
- Java interface static methods are good for providing utility methods, for example null check, collection sorting etc.
- Java interface static method helps us in providing security by not allowing implementation classes to override them.
- We can't define interface static method for Object class methods, we will get compiler error as "This static method cannot hide the instance method from Object". This is because it's not allowed in java, since Object is the base class for all the classes and we can't have one class level static method and another instance method with same signature.

Let’s look into this with a simple example.

```
interface Arithmetic
{
        void add(int x,int y);
        static void sub(int x,int y)
        {
                System.out.println("Substraction = "+(x-y));
        }
}
class Operations implements Arithmetic
{
        public void add(int x, int y)
        {
                System.out.println("Addition = "+(x+y));
        }
}
public class DemoDefaultMethod
{
        public static void main(String[] args)
        {
                Arithmetic obj=new Operations();
                obj.add(45, 30);
                Arithmetic.sub(45, 30);
        }
}
```

Output
Addition = 75
Substraction = 15

Functional Interfaces

Java is an object-oriented programming language i.e. everything in java rotates around the java classes and their objects. No function is independently present on its own in java. They are part of classes or interfaces. And to use them we require either the class or the object of the respective class to call that function. Functional Interface in Java enables users to implement functional programming in Java. In functional programming, the function is an independent entity.

Functional interfaces were introduced in Java 8. A functional interface can contain only one abstract method and it can contain any number of static and default (non-abstract) methods.

An interface with exactly one abstract method is called Functional Interface. @FunctionalInterface annotation is added so that we can mark an interface as functional interface. It is not mandatory to use it, but it's best practice to use it with functional interfaces to avoid addition of extra methods accidentally. If the interface is annotated with @FunctionalInterface annotation and we try to have more than one abstract method, it throws compiler error. The major benefit of java 8 functional interfaces is that we can use lambda expressions to instantiate them and avoid using bulky anonymous class implementation. Java 8 has defined a number of functional interfaces in java.util.function package. Some of the useful java 8 functional interfaces are Consumer, Supplier, Function and Predicate.

Let us understand the functional interfaces with a simple example.

```
@FunctionalInterface
interface Arithmetic
{
        int add(int x,int y);
}

public class DemoFunctionalInterface implements Arithmetic
{
        @Override
        public int add(int x, int y)
        {
                return (x+y);
```

```
    }
    public static void main(String[] args)
    {
        Arithmetic obj=new DemoFunctionalInterface();
        System.out.println("Addition = "+obj.add(45,32));
    }
}
```

Output
Addition = 77

Lambda Expressions

Lambda expression in Java is a feature introduced in Java 8 that allows you to implement one-function interfaces (functional interfaces) more simply and concisely. It is an anonymous function that adds functional programming techniques to Java, making it easier to write code in specific situations compared to using anonymous inner classes.

Java Lambda Expression Syntax

1. No Parameter Syntax

It is the case when the function does not require any input parameter, so don't provide anything in the parameters and the rest of the things are the same.

```
// No Parameter Syntax

() -> {body of function}
```

2. One Parameter Syntax

We only need to pass one input parameter in this situation; however, while using lambda, we don't need to specify the data type of input parameters because the compiler discovers it automatically.

```
// One Parameter Syntax
( p1 ) -> {body of function}
```

3. Multiple Parameter Syntax

The case, as the name implies, refers to a circumstance in which two input parameters are necessary.

```
// Two Parameter Syntax
( p1, p2 ) -> {body of function}
```

Java Lambda Expression Example

```
@FunctionalInterface
interface Arithmetic
{
        int add(int x,int y);
}
public class DemoLambada
{
        public static void main(String[] args)
        {
                Arithmetic obj=(x,y)->(x+y);
                System.out.println("addition = "+obj.add(40,50));
        }
}
```

Method References

Java provides a new feature called method reference in Java 8. Method reference is used to refer method of functional interface. It is compact and easy form of lambda expression. Each time when you are using lambda expression to just referring a method, you can replace your lambda expression with method reference.

Types of Method References

There are following types of method references in java:

1. Reference to a static method.
2. Reference to an instance method.
3. Reference to a constructor.

1. Reference to a static method.

You can refer to static method defined in the class. Following is the syntax and example which describe the process of referring static method in Java.

Syntax:

object::instanceMethod

2. Reference to an instance method.

Like static methods, you can refer instance methods also. In the following example, we are describing the process of referring the instance method.

Syntax:

Class::staticMethod

3. Reference to a constructor.

You can refer a constructor by using the new keyword. Here, we are referring constructor with the help of functional interface.

Syntax:

ClassName::new

Stream API

Java provides a new additional package in Java 8 called java.util.stream. This package consists of classes, interfaces and enum to allows functional-style operations on the elements. You can use stream by importing java.util.stream package.

Stream does not store elements. It simply conveys elements from a source such as a data structure, an array, or an I/O channel, through a pipeline of computational operations. Stream is functional in nature. Operations performed on a stream does not modify its source. For example, filtering a Stream obtained from a collection produces a new Stream without the filtered elements, rather than removing elements from the source collection. Stream is lazy and evaluates code only when required. The elements of a stream are only visited once during the life of a stream. Like an Iterator, a new stream must be generated to revisit the same elements of the source.

You can use stream to filter, collect, print, and convert from one data structure to other etc. In the following examples, we have applied various operations with the help of stream.

Example:

```
import java.util.ArrayList;
```

```
import java.util.List;
import java.util.stream.Collectors;

public class DemoStreamAPI {

	public static void main(String[] args)
	{
		ArrayList<Integer> list=new ArrayList<Integer>();
		list.add(10);
		list.add(11);
		list.add(12);
		list.add(9);
		list.add(8);
	List<Integer> li=list.stream().filter(x->(x%2==0)).collect(Collectors.toList());
		System.out.println(li);
		int max=list.stream().max((x,y)->x>y?1:-1).get();
		System.out.println("Max = "+max);
		long count=list.stream().filter(x->x%2==0).count();
		System.out.println("Count = "+count);
	}
}
```